

ETHEREAL · SECURITY REVIEW

Ethereal

Contract Updates.

A security review of Contract Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Ethereal	Contract Updates	Guardian	September 24 – October 16, 2025	October 23, 2025

§ 00

Contents.

74 pages — overview, confidence ranking, scope, 59 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 56 findings	P. 11
	Remediation Review · 3 findings	P. 70
§ 05	Disclaimer	P. 73
§ 06	About Guardian	P. 74

§ 01

Overview.

Guardian reviewed Contract Updates across 2 review rounds. The final report includes 5 High, 4 Medium, 22 Low, 28 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	meridianxyz/contracts
CLIENT	METHODOLOGY
Ethereal	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Contract Updates	dd90e25d0074
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	f7da9cf95632
AUDIT WINDOW	
September 24 – October 16, 2025	
FINAL REPORT	
October 23, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	5	0	1	0	4
Medium	4	0	2	0	2
Low	22	0	11	0	11
Informational	28	0	8	0	20
All severities	59	0	22	0	37

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Contract Updates in `meridianxyz/contracts`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	ProcessActions Blocked By Malicious Contract	DoS	HIGH	RESOLVED	p. 11
H-02	rescueFunds Fails Due To Missing payable	Logical Error	HIGH	RESOLVED	p. 12
H-03	processActions Blocked By Gas Griefing	Gas Griefing	HIGH	RESOLVED	p. 13
H-04	finalizeWithdrawals Doesn't Work As Expected	Logical Error	HIGH	RESOLVED	p. 14
H-05	Insolvent Accounts Break Balance Accounting	Logical Error	HIGH	ACKNOWLEDGED	p. 15
M-01	finalizeWithdrawals Gas Griefing	Gas Griefing	MEDIUM	RESOLVED	p. 17
M-02	Invalid Signatures Block processActions	DoS	MEDIUM	RESOLVED	p. 18
M-05	realizePnl Gives Traders Infinite Liquidity	Gaming	MEDIUM	ACKNOWLEDGED	p. 19
M-06	newCost Underflow Edge Case DoS	Rounding	MEDIUM	ACKNOWLEDGED	p. 20
L-01	lastWithdrawRequestId Increased Even When Failed	Logical Error	LOW	ACKNOWLEDGED	p. 22
L-03	Lacking _completeWithdrawal Refund Logic	Unexpected Behavior	LOW	RESOLVED	p. 23
L-05	Expired Actions Halt processActions	Warning	LOW	ACKNOWLEDGED	p. 24

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-06	RevokeLinkedSigner Ineffective Expiry Check	Validation	LOW	RESOLVED	p. 25
L-07	Lacking Funding Update Guardrails	Trust Assumptions	LOW	ACKNOWLEDGED	p. 26
L-08	Missing receive In Claimer	Logical Error	LOW	RESOLVED	p. 27
L-10	Off By One Error In ActionDecoder	Unexpected Behavior	LOW	RESOLVED	p. 28
L-11	Unsafe Casting	Validation	LOW	ACKNOWLEDGED	p. 29
L-12	Unexpected OOB Decoding Revert	Validation	LOW	RESOLVED	p. 30
L-13	Removal Misses Pending Withdrawals	Validation	LOW	ACKNOWLEDGED	p. 31
L-14	Lacking Pyth Lazer Channel Validation	Validation	LOW	ACKNOWLEDGED	p. 32
L-15	Mismatch Event Missed For 1 Wei Inaccuracy	Logical Error	LOW	RESOLVED	p. 33
L-16	Missing updateDelegateDepositMaxBatchSize Event	Events	LOW	RESOLVED	p. 34
L-17	Pyth Lazer Refunds Are Stuck	Warning	LOW	ACKNOWLEDGED	p. 35
L-18	Used Nonce Permanently Prevents Signer Revoke	DoS	LOW	ACKNOWLEDGED	p. 36
L-19	destinationAddress Ignored For Native Withdrawal	Logical Error	LOW	RESOLVED	p. 37
L-21	Inexact Balance Updates Perturb Total Balance	Rounding	LOW	ACKNOWLEDGED	p. 38
L-22	Precision Threshold Inconsistency	Unexpected Behavior	LOW	RESOLVED	p. 39
L-23	maintenanceMargin Precision Preservation	Rounding	LOW	RESOLVED	p. 40
L-24	ETH Trapped In Claimer	Unexpected Behavior	LOW	RESOLVED	p. 41

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-01	removeProtected Typo	Documentation	INFORMATIONAL	RESOLVED	p. 42
I-02	finalizeWithdrawals Typo	Documentation	INFORMATIONAL	RESOLVED	p. 43
I-04	Duplicate Proxy Contracts	Best Practices	INFORMATIONAL	RESOLVED	p. 44
I-05	Unnecessary _delegateCallVerifier Function	Superfluous Code	INFORMATIONAL	RESOLVED	p. 45
I-06	Domain Separator Version Cannot Be Changed	Warning	INFORMATIONAL	RESOLVED	p. 46
I-07	finalizeRevokeLinkedSigner Has No Delay	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 47
I-08	Unused Errors	Error	INFORMATIONAL	RESOLVED	p. 48
I-09	Missing Product Id Sanity Check	Validation	INFORMATIONAL	RESOLVED	p. 49
I-10	Max Quantity Is Not Always Abided	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 50
I-11	Liquidations Allowed For Pending Products	Warning	INFORMATIONAL	RESOLVED	p. 51
I-12	Missing lotSize Validation	Validation	INFORMATIONAL	RESOLVED	p. 52
I-13	Liquidator Configuration Check	Validation	INFORMATIONAL	RESOLVED	p. 53
I-14	finalizeRevokeLinkedSigner Ignores Expiry	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 54
I-15	msg.value Allowed For Local Withdrawals	Validation	INFORMATIONAL	RESOLVED	p. 55
I-16	Withdrawals Stuck For Users Due To Wrong Token	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 56
I-17	Small Withdrawals Can Be Stuck	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 57
I-18	PnL Realization Is Not Restricted To Profits	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 58

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-19	Incorrect Error Used For Token Removal	Error	INFORMATIONAL	RESOLVED	p. 59
I-22	Misleading emergencyUnpause Revert	Error	INFORMATIONAL	RESOLVED	p. 60
I-23	Intermediate Overflow Is Not Handled	Best Practices	INFORMATIONAL	RESOLVED	p. 61
I-25	Unnecessary Uint128 Cast	Gas Optimization	INFORMATIONAL	RESOLVED	p. 62
I-26	Unnecessary _revert Return Value	Best Practices	INFORMATIONAL	RESOLVED	p. 63
I-27	Misleading _delegateCallPerpEngine Comment	Documentation	INFORMATIONAL	RESOLVED	p. 64
I-28	Misleading ProductNotFound Revert	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 65
I-29	Unnecessary Addition in Maker Signature Slice	Gas Optimization	INFORMATIONAL	RESOLVED	p. 66
I-30	Settling Must Be Completed In The Correct Order	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 67
I-31	Lacking Address Code Validation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 69

REMEDIATION REVIEW

L-01	Overallocated Refunds Lost	—	LOW	ACKNOWLEDGED	p. 70
L-02	Incorrect Refund Amount in _completeWithdrawal	Logical Error	LOW	ACKNOWLEDGED	p. 71
I-01	Unnecessary MSTORE Operation	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 72

HIGH **H-01**

RESOLVED

ProcessActions Blocked By Malicious Contract

CATEGORY DoS
LOCATION VerifierLib.sol
REVIEW Main Review

DESCRIPTION

The `isValidSignature` function is intended to not revert to allow the `processActions` loop to complete when a `_handleRevokeLinkedSigner` or `_handleLinkSigner` action is performed.

However `isValidSignature` execution can be forced to revert by the sender account address if it houses bytecode and implements a `isValidSignature` function that returns an invalid return payload.

For example, if the `isValidSignature` function returns empty returndata but successfully executes, the `_verifyEip1271Signature` function will still attempt to decode the returndata and revert with an `abi.decode` error.

This will block the `processActions` queue from processing the batch of actions and hold up the exchange.

RECOMMENDATION

Decode the `magicValue` by casting the `returnData` to a `bytes4` object to capture only the first 4 bytes of the `returnData`, and produce an empty `bytes4` object if no `returnData` exists:

```
bytes4 magicValue = bytes4(returnData);
```

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/e18be39dd29bc779443f77f9cf5a58b7>

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/40379fb870ad5d56189068c78412c29597975951>

HIGH **H-02**

RESOLVED

rescueFunds Fails Due To Missing payable

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1758647002440/blob/dd90e25d0074ffa6e1e406d0d6c9a1111be4fedc/src/CollateralManager.sol#L213
REVIEW	Main Review

DESCRIPTION

When a withdrawal remains pending for an extended period, the `rescueFunds` function is invoked to finalize the withdrawal and recover the funds.

However, the `rescueFunds` function is not payable, while the `_completeWithdrawal` function called during this process requires `msg.value`. As a result, funds cannot be rescued if withdrawn via LayerZero OFT.

RECOMMENDATION

Make the `rescueFunds` function payable.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/263>

HIGH **H-03**

RESOLVED

processActions Blocked By Gas Griefing

CATEGORY	Gas Griefing
LOCATION	Todo
REVIEW	Main Review

DESCRIPTION

The `processActions` function may invoke an untrusted external call when it attempts an ERC1271 contract signature validation for the `_handleLinkSigner` or `_handleRevokeLinkedSigner` actions.

This call is protected as much as possible, using `staticcall` and success handling to ensure no reverts can occur. However the amount of gas that is forwarded to the `sender` address is only limited by the 63/64 rule and therefore allows the untrusted `sender` address to expend an enormous amount of gas, leaving an insufficient amount for the completion of the `processActions` loop.

Furthermore, all of the untrusted `sender`'s `returnData` is loaded in, even if a cap is made on the amount of gas that is forwarded to the `sender`, the `sender` address can cause an unexpected amount of gas to be used by loading in a large amount of `returnData`. This can additionally cause unexpectedly high gas usage which may prevent the processing of the `processActions` loop, even if a mere gas forwarding limit is put in place.

RECOMMENDATION

Put in place a limit on the amount of gas forwarded to the `sender` address with the invocation of the `isValidSignature` function. And furthermore, only load in the necessary 4 bytes of `returnData` from the `sender` with a low level call.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/259>

HIGH **H-04**

RESOLVED

finalizeWithdrawals Doesn't Work As Expected

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1758647002440/blob/dd90e25d0074ffa6e1e406d0d6c9a1111be4fed/src/Claimer.sol#L93
REVIEW	Main Review

DESCRIPTION

The `finalizeWithdrawals` function in the Claimer sends `msg.value` inside a for loop. During the first iteration, the entire `msg.value` is sent to the `exchangeGateway`, leaving no balance for subsequent iterations. As a result, only the first withdrawal succeeds, while the remaining ones fail.

RECOMMENDATION

Transfer only the required amount of value in each iteration.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/aa49101f89f48afeffb26d94a45c6f6014b88894>

HIGH **H-05**

ACKNOWLEDGED

Insolvent Accounts Break Balance Accounting

CATEGORY Logical Error
LOCATION Liquidation.sol
REVIEW Main Review

DESCRIPTION

The addition of the `fundingCoverageCharge` indicates that the protocol intends to support insolvent liquidations, particularly in the case where the cumulative funding across all products for the account is greater than the available account usde margin balance. However across many cases, the accounting for this breaks down.

The `userEquityAfterFunding` is determined as the usde token balance of the account onchain less the cumulative funding across all products. This amount is then assigned as the `referenceFundingCoverageCharge` which must match the provided `action.fundingCoverageCharge` within a precision threshold of 13 wei, although the comment purports this to be exact, not requiring this threshold.

In many cases, the user will not be able to cover their funding charges net of PnL, however this will not be captured by the `userEquityAfterFunding` as it does not include negative PnL from positions.

Consider the following example:

- Trader A has 100 USDe available margin balance
- Trader A has -\$80 PnL
- Trader A has a pending funding of \$30 to pay
- Trader A certainly qualifies for the insolvent `fundingCoverageCharge` case, however the computed `userEquityAfterFunding` is \$70 which does not qualify as it is positive

Furthermore, accounts that are liquidated while they have a net profit overstate their equity after funding and deduct more than they should from the liquidator:

- Trader A has 20 USDe available margin balance
- Trader A has \$30 PnL
- Trader A has a pending funding of \$30 to pay
- Trader A can technically cover this funding charge if all is settled, however they still appear as if they are insolvent as measured by the `userEquityAfterFunding` which reports an equity of -\$10

H-05 Insolvent Accounts Break Balance Accounting

CONTINUED

— RECOMMENDATION

One solution to this would be to settle the account PnL right before liquidation every time, however this still comes with rounding and potential price discrepancy concerns.

The funding coverage charge logic should instead be updated to account for the account's net balance factoring in PnL to cover true account insolvency cases when an account cannot cover funding net of their PnL.

— RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/267>

MEDIUM **M-01**

RESOLVED

finalizeWithdrawals Gas Griefing

CATEGORY Gas Griefing
LOCATION CollateralManager.sol
REVIEW Main Review

DESCRIPTION

In the `finalizeWithdrawals` function, the whitelisted `withdrawalClaimers` or the `owner` process batches of withdrawal objects to send these funds to users.

The execution of `finalizeWithdraw` makes an untrusted external call to the account with the `safeTransferUSDe` function.

The `safeTransferUSDe` safely does not load in any `returnData` from the `account` address invocation, however it does forward the entire gas available at the time of the call execution.

As a result, if the `account` address is home to a contract or an EOA that has associated bytecode through EIP 7702 it can expend an unexpected amount of gas and use up the entire 63/64 remaining gas that was forwarded to it.

It is unlikely that the `finalizeWithdrawals` function would be able to complete execution with roughly 1/64th of the initial gas provided, especially if there are other withdrawals to process. So this results in a DoS of the `finalizeWithdrawals` loop for any withdrawal batches that would include malicious actions.

RECOMMENDATION

Create a version of the `safeTransferUSDe` function that only forwards a configured safe gas amount so that the account cannot gas grief the executor and potentially halt the processing of other withdrawals.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/268>

MEDIUM **M-02**

RESOLVED

Invalid Signatures Block processActions

CATEGORY DoS
LOCATION VerifierLib.sol
REVIEW Main Review

DESCRIPTION

For EOA signature validation the `ECDSA.recover` function from the Solady library is used to verify the signer of the message, however this recover function reverts upon receiving malformed signatures.

The `verifierLib.isValidSignature` function intends to not revert but instead return a boolean indicating successful verification to prevent stopping the `processActions` loop.

As a result if a malformed signature is provided to the signer update actions this can result in a block of the `processActions` loop.

RECOMMENDATION

Instead of `ECDSA.recover`, use the `ECDSA.tryRecover` function from the Solady library.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/6281999078fc72b0d8f6864f4e929299f0ceb75e>

MEDIUM **M-05**

ACKNOWLEDGED

realizePnl Gives Traders Infinite Liquidity

CATEGORY Gaming
LOCATION PerpEngine.sol: 302
REVIEW Main Review

— DESCRIPTION

In the `realizePnl` function the trader's pending unrealized PnL is moved into their account balance without having to be matched against other orders on the orderbook. This allows a trader to realize their on-paper gains on a perp position without experiencing any market impact.

In some cases with very large positions or especially thin order books near the market price, a trader may net more from the (realizePnl action - initial margin) than compared to actually exiting their position and withdrawing the resulting usde balance that remains after experiencing price impact in the market.

— RECOMMENDATION

When realizing the PnL for a trader with the `realizePnl` function, be sure to only realize profit to a conservative market price, taking into account the current market price and the orderbook depth relative to the size of the position being realized.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

newCost Underflow Edge Case DoS

CATEGORY Rounding
LOCATION Liquidation.sol
REVIEW Main Review

DESCRIPTION

In the FLIP case in the `_settleLiquidatorPosition` function the `newCost` is calculated based on:

```
newCost = eTransferredNotional - (eLiquidatorRealizedPnl * oldSideSign) - cOldCost
```

Which is then cast to a `uint` value. However in some cases, where the flipped amount is small this can lead to an underflow revert. Consider the following scenario, which requires a small `lotSize`.

Index token price is \$5. The liquidator has a 100,000 index token long position, the cost is \$500,000

Liquidated Position collateral = 2505 USDe

Liquidated Position size = 100000e9 + 1, short

Liquidated Position Notional = (100000e9 + 1) * \$5 = 5000000000000005

Liquidated Position cost = 5000000000000005

maxLeverage = 100x

Margin factor = 0.5x

The index token price stays at \$5

A Funding of \$10 is charged

The position is left with:

Position Notional = (100e9 + 1) * 1,000 = 100000000001000

Pending PnL = 0

Net Margin = 2495e9

Maintenance margin is 5000000000000005 * 1/200 = 25000000000000

Liquidation Occurs

M-06 newCost Underflow Edge Case DoS

CONTINUED

```
rt.oldSize = 100000e9
rt.newSize = -1
rt.quantity = 100000e9 + 1
eTransferredNotional = (100000e9 + 1) * 5 + 2495e9 = 502495000000005
cOldCost = 500000e9
cCloseNotional = 100000e9 * 502495000000005 / (100000e9 + 1) = 502494999999999
oldSideSign = 1
eLiquidatorRealizedPnl = 502494999999999 - 500000e9 = 249499999999 (with exact accuracy to
the contracts reference)
newCost = 502495000000005 - (249499999999) - 500000e9 = 6
```

The `eLiquidatorRealizedPnl` inaccuracy threshold is however `(eTransferredNotional >> 52) + 13`, which is trivially larger than 6, thus allowing the `newCost` resulting value to often be negative in this case and resulting in a casting underflow revert.

Not only will this revert prevent the liquidation from occurring, but it will also halt the batch that is actively being processed.

RECOMMENDATION

Firstly, do not configure products with a `lotSize` less than an order of magnitude of 100 wei to be safe and consider adding validation at the contract level to enforce this configuration.

Furthermore, consider flooring the `newCost` to zero in the event that this case does arise to avoid harmful reverts.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1758647002440/pull/1/files>

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

lastWithdrawRequestId Increased Even When Failed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1758647002440/blob/f2a1ba4cca526f52c0748ed8e56371d207cdb1cc/src/Claimer.sol#L106
REVIEW	Main Review

— DESCRIPTION

The `finalizeWithdrawals` function in the Claimer contract advances `lastWithdrawRequestId` to the last ID in the batch, even if some withdrawals in the batch fail. Since the function expects IDs to be strictly increasing, the failed withdrawals cannot be retried by the claimer.

— RECOMMENDATION

Consider tracking the IDs of failed withdrawals and allowing the claimer to retry them if necessary.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

Lacking `_completeWithdrawal` Refund Logic

CATEGORY Unexpected Behavior
LOCATION CollateralManager.sol: 314
REVIEW Main Review

DESCRIPTION

In the `_completeWithdrawal` function there is validation to ensure that the provided `msg.value` is greater than or equal to the necessary `messagingFee.nativeFee` fee. However there is no logic to refund the `msg.sender` in the event that the provided `msg.value` is greater than the necessary `messagingFee.nativeFee`.

The layerzero `oft.send` call does use the `msg.sender` as the refund recipient, however this `oft.send` call will not provide any refund ever since the forwarded value is simply the `messagingFee.nativeFee + amountLD`.

RECOMMENDATION

Implement refund logic at the end of the `_completeWithdrawal` function that refunds the excess `msg.value`, notice that the `oft.send` refund cannot be used since it requires exact payment.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/265>

LOW **L-05**

ACKNOWLEDGED

Expired Actions Halt processActions

CATEGORY	Warning
LOCATION	ActionHandler.sol
REVIEW	Main Review

— DESCRIPTION

In the `exchange.verifyBeforeActionExpiry` function, if the expiry of the action has come to pass then the operation reverts, blocking subsequent action processing.

This is in contrast to other areas where non-reverting failures are preferred to allow other actions to continue processing such as invalid signatures with the `isSignatureValid` function.

Be aware that it may be possible for actors to block stuff for a short period of time to delay a `processActions` batch until one of these expiries are breached.

— RECOMMENDATION

Consider if actions should silently be skipped or revert the whole `processActions` batch if they are expired. This has implications for how an action might affect others in the batch if they are unsuccessful.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

RevokeLinkedSigner Ineffective Expiry Check

CATEGORY Validation
LOCATION ActionHandler.sol
REVIEW Main Review

DESCRIPTION

The `verifyBeforeActionExpiry` validation comes after the failed signature validation early return case where the `pendingSignerRevoke` entry is assigned to true. As a result, signatures that are made invalid by the sender after sending them, such as a gnosis safe consuming the related nonce, can be marked as pending in the `pendingSignerRevoke` mapping.

These actions cannot be replayed due to the nonce check in the `revokeSigner` function, however to avoid any unexpected behavior the expiry validation should be carried out before the invalid signature early return case.

RECOMMENDATION

Move the `verifyBeforeActionExpiry` validation to before the `!isSignatureValid` case handling.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/265>

LOW **L-07**

ACKNOWLEDGED

Lacking Funding Update Guardrails

CATEGORY Trust Assumptions
LOCATION PerpEngine.sol
REVIEW Main Review

— DESCRIPTION

In the `updateFunding` function there are no guardrails which prevent the sequencer from calling the function with a completely invalid `fundingDelta` that either awards users with a large sum of USDe, or causes them to immediately become insolvent. This may be a mechanism through which a compromised sequencer account drains the funds in the `ExchangeGateway` contract.

— RECOMMENDATION

Be aware of this risk and consider adding guardrails to how much the `fundingDeltaUsd` can change over time.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

RESOLVED

Missing receive In Claimer

CATEGORY Logical Error
LOCATION Claimer.sol
REVIEW Main Review

DESCRIPTION

During the `finalizeWithdrawals` flow, the `Claimer` sends `msg.value` to the `exchangeGateway`, but this value is not based on the LayerZero quote. This means the Claimer must send more than enough to the `exchangeGateway`. However, the `exchangeGateway` transfers only the exact required amount to LayerZero, and the excess is never refunded to the Claimer.

Additionally, the Claimer does not implement a receive function to accept refunds, even if a refund mechanism were introduced.

RECOMMENDATION

Add a receive function to the Claimer contract and refund any excess back to the Claimer.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/265>

LOW **L-10**

RESOLVED

Off By One Error In ActionDecoder

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/contracts-team1-1758647002440/blob/dd90e25d0074ffa6e1e406d0d6c9a1111be4fedc/src/lib/ActionDecoder.sol#L161-L167>

REVIEW Main Review

DESCRIPTION

When `action.length == MATCH_ORDER_TAKER_SIGNATURE_LENGTH`, the if condition below evaluates to false and the code proceeds. However, accessing `action[MATCH_ORDER_TAKER_SIGNATURE_LENGTH]` is out of bounds, since valid indices are `0..action.length-1`. This results in a panic error instead of reverting with the intended `InvalidActionEncoding` custom error.

```
// There must be enough data to read both signature lengths
if (action.length < MATCH_ORDER_TAKER_SIGNATURE_LENGTH) {
    (ActionId actionId, ActionType actionType) = decodeActionHeader(action);
    revert InvalidActionEncoding(actionId, actionType);
}

uint16 makerSignatureLength = uint8(action[MATCH_ORDER_MAKER_SIGNATURE_LENGTH]);
uint16 takerSignatureLength = uint8(action[MATCH_ORDER_TAKER_SIGNATURE_LENGTH]);
```

RECOMMENDATION

Revert if `action.length <= MATCH_ORDER_TAKER_SIGNATURE_LENGTH`

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/265>

LOW **L-11**

ACKNOWLEDGED

Unsafe Casting

CATEGORY	Validation
LOCATION	PerpEngine.sol
REVIEW	Main Review

DESCRIPTION

Throughout the Ethereum exchange casts are made without the use of `SafeCastLib`.

In particular the cast of the `currentCost` to `int128` is at risk of overflow in the `realizePnL` function in the rare case where the `currentCost` exceeds the `int128.max` value.

```
int128 newCost = int128(currentCost) + costDelta;
```

RECOMMENDATION

Use `SafeCast` for this cast in the `realizePnL` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/273>

LOW **L-12**

RESOLVED

Unexpected OOB Decoding Revert

CATEGORY Validation
LOCATION ActionDecoder.sol
REVIEW Main Review

DESCRIPTION

When `action.length < 9`, `decodeMatchOrdersAction` calls `decodeActionHeader` to populate `InvalidActionEncoding`, but `decodeActionHeader` reads `bytes8(action[:8])` and `action[8]`, which will revert with an out-of-bounds panic. This produces an unexpected generic revert rather than the intended `InvalidActionEncoding` error, complicating upstream error handling.

RECOMMENDATION

Before calling `decodeActionHeader` for error reporting, first check `action.length >= ACTION_HEADER_OFFSET` (9). If shorter, revert `InvalidActionEncoding` with zeroed id/type or a dedicated error that does not attempt to parse the header.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/273>

LOW **L-13**

ACKNOWLEDGED

Removal Misses Pending Withdrawals

CATEGORY Validation
LOCATION ExchangeConfig.sol
REVIEW Main Review

DESCRIPTION

The `removeToken` function only checks `Account.DataGlobal.totalBalance` and `totalPendingBalance`, but does not verify whether any withdrawals for the token are pending. Removing a token zeroes its `Token.Data` (including `tokenAddress` and `oftAddress`), which can cause `finalizeWithdraw` to lack required metadata (e.g., ERC20 address or OFT adapter) and revert or behave incorrectly for already-initiated withdrawals.

RECOMMENDATION

Before allowing removal, also verify there are no pending withdrawals for the token across all accounts, perhaps by maintaining this count in storage per token.

RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/273/files>

LOW **L-14**

ACKNOWLEDGED

Lacking Pyth Lazer Channel Validation

CATEGORY	Validation
LOCATION	Liquidation.sol
REVIEW	Main Review

— DESCRIPTION

In the `_parsePythPriceFeed` function there is no validation that confirms the channel that is being used for the Pyth lazer price.

The Pyth Lazer example project includes a channel validation to validate that the channel is the RealTime channel.

— RECOMMENDATION

Consider adding an explicit validation at the contract level that the channel is as expected and matches the RealTime channel:

```
if (channel != PythLazerLib.Channel.RealTime) {  
  revert("expected update from RealTime channel");  
}
```

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

RESOLVED

Mismatch Event Missed For 1 Wei Inaccuracy

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Throughout the PerpEngine and Liquidation contracts the verification functions which assess the tolerance difference between the provided offchain value and the value computed onchain emit an event when there is a mismatch between the offchain and onchain values that is below the tolerance threshold and thus acceptable.

However if the mismatch is 1 wei then no event is emitted, even though a mismatch was found, this is because the event emission case always uses `else if (absDiff > 1)`.

— RECOMMENDATION

Compare the `absDiff` to see if it is greater than zero: `else if (absDiff > 0)`.

— RESOLUTION

Status: Resolved.

LOW **L-16**

RESOLVED

Missing updateDelegateDepositMaxBatchSize Event

CATEGORY Events
LOCATION ExchangeConfig.sol: 362
REVIEW Main Review

DESCRIPTION

In the `updateDelegateDepositMaxBatchSize` function there is no event emitted to denote the update as described in the `README` and as is done in other configuration functions.

RECOMMENDATION

Consider adding an event to signal the update made in the `updateDelegateDepositMaxBatchSize` function.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/273>

LOW **L-17**

ACKNOWLEDGED

Pyth Lazer Refunds Are Stuck

CATEGORY Warning
LOCATION Liquidation.sol: 217
REVIEW Main Review

— DESCRIPTION

In the liquidation flow the pythFee is forwarded to the pythLazer contract which refunds excess native that might have been sent. There is however no logic to process refunds that may have been sent back by Pyth Lazer on the liquidation action.

— RECOMMENDATION

Consider adding a function that allows the contract owner to sweep any native assets which may have been refunded.

— RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

Used Nonce Permanently Prevents Signer Revoke

CATEGORY DoS
LOCATION ActionHandler.sol: 372
REVIEW Main Review

DESCRIPTION

In the `_handleRevokeLinkedSigner` function the `pendingSignerRevoke` mapping entry for the subaccount and signer is set as `true` when the signature is invalid as reported by the `isValidSignature` function. When this case occurs, the `revokeSigner` function will be called in the `finalizeRevokeLinkedSigner` function to clear this pending withdrawal.

While the pending withdrawal is present, the signer cannot be revoked for that subaccount with any other signature.

As a result, if a message is ever submitted that uses a nonce that was already consumed for the `RevokeLinkedSigner` action type, then the revocation will be stuck forever, as the `finalizeRevokeLinkedSigner` also validates that the nonce was not previously used in the `revokeSigner` function.

RECOMMENDATION

Since the signature verification is being skipped in the `finalizeRevokeLinkedSigner` function, consider if the nonce verification can be skipped in this instance as well, while still marking it idempotently as used after the finalization. Otherwise ensure that this case can never arise through the sequencer.

RESOLUTION

Status: Acknowledged.

LOW **L-19**

RESOLVED

destinationAddress Ignored For Native Withdrawal

CATEGORY	Logical Error
LOCATION	CollateralManager.sol
REVIEW	Main Review

DESCRIPTION

In the `_completeWithdrawal` function when withdrawals are made using the native L3 chain instead of OFT withdrawals, the `withdraw.destinationAddress` is ignored and the withdrawal is always sent to the account.

However for some accounts, especially those that use the EIP 1271 integration, they may not expect to receive the USDe funds and are expecting them to go to the `destinationAddress` they specified.

In some cases this could end up trapping USDe for account addresses that cannot handle it.

RECOMMENDATION

Either honor the `withdrawal.destinationAddress` during the `_completeWithdrawal` flow for native chain withdrawals, or explicitly validate that the `destinationAddress` is not configured for withdrawals that will use a `destinationEndpointId` of `0` or the exchange `lzEndpointId`.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/271>

LOW **L-21**

ACKNOWLEDGED

Inexact Balance Updates Perturb Total Balance

CATEGORY	Rounding
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Throughout the Exchange contracts, in liquidation and order matching, the expected pnl of the action is returned from the settlement action. The expected pnl of the action can however have a discrepancy of a handful of wei, this can cause drift from the totalBalance recorded, breaking the invariant that the sum of user balances is the same as the totalBalance.

This drift from the totalBalance can cause issues when removing a token or when the final depositor for a token is attempting to withdraw.

Firstly, it may not be possible for the totalBalance to reach zero due to the drift between the individual balances entries. Notice that the `removeToken` function validates that the `totalBalances` mapping is exactly 0 for the token before carrying out the removal.

Secondly, a final depositor may encounter a revert when attempting to withdraw all of their balance due to an underflow when deducting the withdrawal amount from the totalBalance. Or the fee claimer may experience a revert when attempting to claim their final fee amount.

— RECOMMENDATION

Consider accounting for these last depositor/shutdown cases by maintaining track of the number of accounts with deposits for a token. If that amount of accounts would become zero, then reset the totalBalances mapping entry for the token to zero.

— RESOLUTION

Status: Acknowledged.

LOW **L-22**

RESOLVED

Precision Threshold Inconsistency

CATEGORY	Unexpected Behavior
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The `_verifyFillNotional` function performs precision validation on the fill notional amount with a tolerance of $(\text{internalValue} \gg 52) + 1$.

However elsewhere in the liquidation transferred notional validation, the imprecision validation is performed with $(\text{eTransferredNotional} \gg 52) + 13$.

This yields inconsistent precision thresholds in these cases and across the codebase which may result in unexpected tolerance invalidations if the offchain system is meeting the lower precision threshold across the board.

RECOMMENDATION

Consider standardizing on a precision constant, or document why these cases deviate from each other.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/1c4b64eda53fad0b77777fc8892da042125c184f>

LOW **L-23**

RESOLVED

maintenanceMargin Precision Preservation

CATEGORY Rounding
LOCATION Liquidation.sol: 232
REVIEW Main Review

DESCRIPTION

In the `liquidateSubaccount` function, the calculation of the maintenance margin divides the `MAINTENANCE_MARGIN_RATIO` by the `product.maxLeverage` before ultimately performing the multiplication with `markNotionalX18`.

This does not favor the highest precision possible and may contribute to some precision inaccuracies and even reverts outside of the prevision tolerance threshold.

RECOMMENDATION

Consider refactoring the `maintenanceMargin` computation such that it maintains as much precision as it possible can.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/275>

LOW **L-24**

RESOLVED

ETH Trapped In Claimer

CATEGORY	Unexpected Behavior
LOCATION	Claimer.sol
REVIEW	Main Review

— DESCRIPTION

Because the Claimer contract does not revert but continues in the loop for any failed withdrawal finalizations, the ETH that was intended to be sent to these finalizations may become trapped in the Claimer contract if not carefully refunded.

— RECOMMENDATION

Along with the fix to H-04, consider adding a refund mechanism to refund the caller for any ETH that went unused due to a failed withdrawal finalization.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/276>

INFORMATIONAL **I-01**

RESOLVED

removeProtected Typo

CATEGORY Documentation
LOCATION Token.sol: 12
REVIEW Main Review

— DESCRIPTION

In the comment for the removeProtected variable there is a typo which confuses the purpose of removeProtected: `/// If the token is used by a product (as either a quote or base asset), it cannot never be removed`.

When in fact the product cannot `ever` be removed.

— RECOMMENDATION

Correct the comment to `/// If the token is used by a product (as either a quote or base asset), it cannot ever be removed`.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL I-02

RESOLVED

finalizeWithdrawals Typo

CATEGORY Documentation
LOCATION Claimer.sol: 99
REVIEW Main Review

DESCRIPTION

In the Claimer finalizeWithdrawals function finalize is misspelled as finialize.

RECOMMENDATION

Correct it to finalize.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL **I-04**

RESOLVED

Duplicate Proxy Contracts

CATEGORY	Best Practices
LOCATION	EtherealProxy.sol;EtherealClaimerProxy.sol
REVIEW	Main Review

DESCRIPTION

The project includes two proxy contracts, `EtherealProxy.sol` and `EtherealClaimerProxy.sol`. Both contracts are identical implementations of `ERC1967Proxy`. Maintaining duplicate proxy contracts adds unnecessary complexity to the codebase.

RECOMMENDATION

Use a single proxy contract to avoid duplication. If two proxies are needed, clearly document their different roles.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL **I-05**

RESOLVED

Unnecessary `_delegateCallVerifier` Function

CATEGORY Superfluous Code
LOCATION ActionHandler.sol
REVIEW Main Review

DESCRIPTION

In the `ActionHandler` contract the `_delegateCallVerifier` is unused as the verifier has been turned into a library.

RECOMMENDATION

Remove the `_delegateCallVerifier` function.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL I-06

RESOLVED

Domain Separator Version Cannot Be Changed

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The `_hashTypedData` function in the Solady `EIP712` library only re-computes the domain separator if the `_domainNameAndVersionMayChange` function returns true.

However the Ethereum Exchange does not override and return true for the `_domainNameAndVersionMayChange` function, therefore when the version of the exchange is bumped, or if the name of the exchange for the domain separator is changed then the corresponding signing domain will not be updated. This is actually the correct behavior, as an updated signing domain would change the resulting `withdrawDigest` that keys pending withdrawals and cause them to be zero'd out.

As a result, the initial value that is provided as the `VERSION` constant must always remain as the version once in production.

RECOMMENDATION

Be aware of this behavior and do not plan on updating the `VERSION` constant as 1) it will not update the signing domain due to a lacking override of `_domainNameAndVersionMayChange` and 2) even if this override is made, it would be catastrophic to update the `VERSION` as it would re-key pending actions.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/273>

INFORMATIONAL **I-07**

ACKNOWLEDGED

finalizeRevokeLinkedSigner Has No Delay

CATEGORY Validation
LOCATION ActionHandler.sol: 118
REVIEW Main Review

DESCRIPTION

The `finalizeRevokeLinkedSigner` function has no delay applied to the validation performed in the function.

As a result a signer could be immediately revoked by the owner and sequencer without providing a valid signature from the user.

RECOMMENDATION

Consider if this should be allowed on the exchange without a delay, and if not, consider adding delay validation similar to the rescue funds function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

RESOLVED

Unused Errors

CATEGORY	Error
LOCATION	Errors.sol
REVIEW	Main Review

— DESCRIPTION

In the `Errors` library there exist the `WithdrawFound` error is unused.

— RECOMMENDATION

Consider removing the `WithdrawFound` error or moving it into the `DeprecatedErrors` library as needed.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL I-09

RESOLVED

Missing Product Id Sanity Check

CATEGORY Validation
LOCATION PerpEngine.sol: 270
REVIEW Main Review

DESCRIPTION

In the `realizeFunding` function the provided list of `productIds` is iterated over and `PerpProduct.load(productIds[i])` is used to load in the relevant product storage pointer.

On the following line the `productId` is assigned to the `product.id` from the storage pointer, however there is no validation performed to ensure as a sanity check that the resulting `productId` is the same as the `productIds` entry of the current index.

RECOMMENDATION

Consider adding a validation to ensure that the declared `productId == productIds[i]`.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Max Quantity Is Not Always Abided

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the PerpEngine contract the `_verifyOrder` function validates that the `order.quantity` is not above the product's specified max quantity. However the max quantity can be surpassed by an order which assigns a quantity of 0 and is treated as a full position close.

In the case where the position size exceeds the product's specified `maxQuantity`, this order can be larger than expected. Especially if matched with another 0 quantity order for a position of the opposite side, this can allow for unexpectedly large matches to occur.

Furthermore, during liquidation there is no validation of the `sizeDelta` against the `maxQuantity`. Though this is expected, this finding notes that the `maxQuantity` does not apply to liquidations.

— RECOMMENDATION

Be aware that the `maxQuantity` is not respected for orders with quantity zero that close the entire existing position, or liquidations.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

RESOLVED

Liquidations Allowed For Pending Products

CATEGORY Warning
LOCATION Liquidation.sol
REVIEW Main Review

DESCRIPTION

In the Liquidation flow there is now validation against the status of the product being liquidated, as a result liquidations could technically be carried out for products that are not currently live.

RECOMMENDATION

Be aware of this possibility and consider explicitly validating that a product is in the **ACTIVE** status before carrying out the liquidation.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/264>

INFORMATIONAL **I-12**

RESOLVED

Missing lotSize Validation

CATEGORY	Validation
LOCATION	PerpEngine.sol
REVIEW	Main Review

DESCRIPTION

In the `_verifyProductSizes` function there is no validation that explicitly prevents the `lotSize` from being 0.

This will result in unexpected and unclear panic division by 0 reverts when validating modulo `lotSize`.

RECOMMENDATION

Consider adding an explicit `lotSize` nonzero validation before the modulo validation.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/279>

INFORMATIONAL I-13

RESOLVED

Liquidator Configuration Check

CATEGORY	Validation
LOCATION	Liquidation.sol
REVIEW	Main Review

DESCRIPTION

In the `liquidateSubaccount` function it may be beneficial to add a liquidator configuration check that ensures that the liquidator account and subaccount are not zero values.

RECOMMENDATION

Consider adding a liquidator account and subaccount address to confirm the liquidator is properly configured as a sanity check.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/273>

INFORMATIONAL **I-14**

RESOLVED

finalizeRevokeLinkedSigner Ignores Expiry

CATEGORY Unexpected Behavior
LOCATION ActionHandler.sol
REVIEW Main Review

DESCRIPTION

The `finalizeRevokeLinkedSigner` function does not validate that the action expiry has not passed at the time of finalization, this may be unexpected for a user if one of their pending revoke actions is executed by the owner long after their action has technically expired.

RECOMMENDATION

This behavior may be acceptable, however be aware of this nuance and consider documenting it for users.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/278>

INFORMATIONAL **I-15**

RESOLVED

msg.value Allowed For Local Withdrawals

CATEGORY	Validation
LOCATION	CollateralManager.sol
REVIEW	Main Review

DESCRIPTION

In the local network withdrawal flow there is no usage of msg.value although it is used within a payable function flow.

RECOMMENDATION

To avoid any chance of potentially trapping Ether value by calling the complete withdrawal flow with ether value for native chain withdrawals, consider reverting if the msg.value is nonzero in the native withdrawal case in `_completeWithdrawal`.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-16**

ACKNOWLEDGED

Withdrawals Stuck For Users Due To Wrong Token

CATEGORY Warning
LOCATION CollateralManager.sol: 268
REVIEW Main Review

DESCRIPTION

In the `_completeWithdrawal` function when the `withdrawToken` does not match the `usdToken` and the destination chain is different from the native network, the function reverts with the `InvalidDepositToken` error.

This pending withdrawal cannot be cleared by either `finalizeWithdrawal` or `rescueFunds`, and therefore holds the account's pending withdrawal balance hostage.

RECOMMENDATION

For now, only the `usdToken` in the balance mapping is supported, so this may be acceptable currently. However in the future consider validating that the pending withdrawal will not fail this validation upon creation to avoid trapping the withdrawal for users.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

Small Withdrawals Can Be Stuck

CATEGORY	Warning
LOCATION	CollateralManager.sol
REVIEW	Main Review

— DESCRIPTION

In the withdrawal flow it's possible for a user to specify a withdrawal amount that is larger than the required withdrawal fee, but rounds to a cross-chain transfer amount of zero due to the decimalConversionRate flooring.

For example:

- The withdrawal fee is 1e9
- User A initiates a withdrawal USDe of $1e9 + 1$ in normalized amounts
- This passes the withdrawal fee validation, the amountMinusFee is 1 wei in normalized form
- On withdraw finalization, the denormalized amount is 1e9
- The amountLd value rounds to zero due to the decimalConversionRate being 1e12
- The amount sent is ultimately 0 value and this results in a revert

Such pending withdrawals cannot be processed and will always remain stuck.

— RECOMMENDATION

Be aware of this and consider validating that the remaining amount after fees is greater than or equal to the decimalConversionRate after being denormalized when a cross-chain transfer is to be executed.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

ACKNOWLEDGED

PnL Realization Is Not Restricted To Profits

CATEGORY Warning
LOCATION PerpEngine.sol: 302
REVIEW Main Review

— DESCRIPTION

In the `realizePnl` function there is no validation that ensures that the `costDelta` being realized as supposed profits to the account's liquid margin balance in the `accountData.balance[subaccount][usdToken]` entry are actually from position profits.

The `costDelta` could be putting the position into extreme losses and allowing the balance mapping to be inflated with profits that the account has not actually received.

It is up to the sequencer to carefully invoke the `realizePnl` function with a cost that represents the actual current market price and the position's actual equity value.

— RECOMMENDATION

Be aware of this lack of validation at the Smart Contract level and ensure that the sequencer currently and in future iterations considers the current profit of positions carefully when assigning the cost for the profit realization.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-19**

RESOLVED

Incorrect Error Used For Token Removal

CATEGORY	Error
LOCATION	ExchangeConfig.sol
REVIEW	Main Review

DESCRIPTION

When total balances are non-zero, `removeToken` reverts with `P_ERR_AMT_ZERO`, which semantically indicates an 'amount zero' error. However the issue is that the token balance is non-zero rather than zero.

RECOMMENDATION

Consider creating a `P_ERR_AMT_NON_ZERO` error or using the `P_ERR_BAD_VAL` error here.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-22

RESOLVED

Misleading emergencyUnpause Revert

CATEGORY Error
LOCATION ExchangeGateway.sol
REVIEW Main Review

DESCRIPTION

The `emergencyUnpause` function reverts with the `ExchangePaused` error in the case that the owner calls the function when the exchange is not paused.

This error may be misleading to the caller in the event that it is surfaced, since it claims that the failure is due to the fact that the exchange is paused when in fact it is due to the fact that the exchange is not paused.

RECOMMENDATION

Consider introducing an `ExchangeUnpaused` error for this case.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-23

RESOLVED

Intermediate Overflow Is Not Handled

CATEGORY Best Practices
LOCATION PerpEngine.sol
REVIEW Main Review

DESCRIPTION

In the partial reduction case of the `_settlePosition` function, the `cFillData.quantity` and `cOldCost` are multiplied together using uint128 operations.

In some rare edge case scenarios this could be at risk of overflowing the maximum `uint128` value, however this is unlikely.

RECOMMENDATION

Out of an abundance of caution, consider using a Math library `mulDiv` function that handles intermediate overflow for the calculation of the `cReduceCost` value.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/275>

INFORMATIONAL I-25

RESOLVED

Unnecessary Uint128 Cast

CATEGORY Gas Optimization
LOCATION Liquidation.sol
REVIEW Main Review

DESCRIPTION

In the partial reduction case of the `_settleLiquidatorPosition` function the `rt.quantity` is casted to a `uint128` to deduct from the `product.openInterest`. However the `rt.quantity` is already a `uint128` and therefore does not need to be casted as such.

RECOMMENDATION

Remove the unnecessary `uint128` cast.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-26

RESOLVED

Unnecessary `_revert` Return Value

CATEGORY	Best Practices
LOCATION	ExchangeGateway.sol
REVIEW	Main Review

DESCRIPTION

The `_revert` function specifies a `string memory` return value, yet never returns any data and instead always reverts.

RECOMMENDATION

Remove the `string memory` return value from the `_revert` function.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-27

RESOLVED

Misleading `_delegateCallPerpEngine` Comment

CATEGORY Documentation
LOCATION ExchangeGateway.sol
REVIEW Main Review

DESCRIPTION

The documentation for the `_delegateCallPerpEngine` function states that it `Performs a delegate call to the ExchangeConfig with the data argument`.

However, it instead makes a delegate call to the `PerpEngine` contract.

RECOMMENDATION

Correct the documentation for the `_delegateCallPerpEngine` function.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-28

RESOLVED

Misleading ProductNotFound Revert

CATEGORY Unexpected Behavior
LOCATION PerpEngine.sol: 273
REVIEW Main Review

DESCRIPTION

In the `realizeFunding` function when an entry in the `productIds` array does not map to an existing configured product, the function reverts with the `ProductNotFound(id, productId, EngineType.PERP)` error.

However the `productId` here is assigned as the resulting memory struct's `product.id`. That corresponds to the `productIds` entry. This product id will always be zero and therefore does not allow the consumer of the `ProductNotFound` error to debug which `productIds` entry failed the validation.

RECOMMENDATION

Correct the error to revert with `ProductNotFound(id, productIds[i], EngineType.PERP)`.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-29

RESOLVED

Unnecessary Addition in Maker Signature Slice

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/contracts-team1-1758655871926/blob/dd90e25d0074ffa6e1e406d0d6c9a1111be4fedc/src/lib/ActionDecoder.sol#L174>

REVIEW Main Review

DESCRIPTION

In the `decodeMatchOrdersAction` function, the decoding of `makerSignature` currently uses the boundary expression:

```
makerSignature: bytes(action[MATCH_ORDER_MAKER_SIGNATURE:MATCH_ORDER_MAKER_SIGNATURE + makerSignatureLength]),
```

However, the value of `MATCH_ORDER_MAKER_SIGNATURE + makerSignatureLength` is already computed and stored in the variable `takerSignatureIndex`. Recomputing the addition introduces unnecessary repetition and slightly reduces code clarity.

RECOMMENDATION

Use the precomputed variable `takerSignatureIndex` when decoding `makerSignature` to improve readability and consistency:

```
makerSignature: bytes(action[MATCH_ORDER_MAKER_SIGNATURE:takerSignatureIndex]),
```

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/274>

INFORMATIONAL I-30

ACKNOWLEDGED

Settling Must Be Completed In The Correct Order

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

In the `_setPosition` function in the order match flow as well as the `realizeFunding` in the funding realization flow, the funding for the account is computed and settled into the account balance mapping. The balance mapping stores values as a uint, meaning that an account's liquid margin balance cannot go below zero.

However there can arise situations where the funding amount exceeds the liquid margin of an account, for example consider the following scenario:

- Trader A holds a 20 ETH long with an entry price of \$5,000 and cost basis of \$100,000
- Trader A has 1,000 USDe of liquid margin, opening their position at an initial leverage of 100x
- The price of ETH goes to \$6,000, the profit of Trader A's position is now \$20,000 making their total margin value \$21,000
- Trader A incurs a funding charge of \$1,250, however the `realizeFunding` and `_setPosition` functions will attempt to deduct this \$1,250 from the Trader's liquid margin balance of just 1,000 USDe, which results in an underflow panic revert

As a result, these actions cannot take place for traders in such a scenario, and result in a DoS of the `processActions` loop.

Similarly, the settling of PnL can also produce an underflow revert when attempting to settle it into the balances mapping if the PnL from other products and funding from other products are not settled beforehand.

RECOMMENDATION

Ensure that `realizePnl` and `realizeFunding` are invoked as needed in the correct order to allow an account's pending PnL and pending funding amounts to fit into the account balance mapping in a positive integer and allow order matches to go through.

I-30 Settling Must Be Completed In The Correct Order

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-31

ACKNOWLEDGED

Lacking Address Code Validation

CATEGORY Validation
LOCATION ExchangeGateway.sol
REVIEW Main Review

DESCRIPTION

In the `ExchangeGateway` contract actions use the `_delegatecall` internal function to delegatecall the relevant `AddressRegistry` record address.

The `_delegatecall` function checks the returned `success` boolean and reverts if a call was unsuccessful.

However the `target.delegatecall(data)` operation will return true always for addresses which are EOAs and do not implement any code at their address.

Therefore, in the event of a misconfiguration in the `AddressRegistry`, if a record is assigned to an incorrect address that happens to be an EOA, the `_delegatecall` function will not revert.

Many actions in the `ExchangeGateway` contract that `delegatecall` to a record are payable and pass along ether to the target address. Therefore in the event of a misconfiguration this Ether could be accidentally trapped.

RECOMMENDATION

Consider adding a code length validation to ensure that the `target` address to in the `_delegatecall` function has bytecode.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Overallocated Refunds Lost

CATEGORY —
LOCATION Claimer.sol
REVIEW Remediation Review

DESCRIPTION

In the `CollateralManager` contract `_completeWithdrawal` function there are several ways that excess native `msg.value` is refunded to the caller.

If the withdrawal is a native chain withdrawal and does not require native funds to be sent then the entire `msg.value` is refunded. Furthermore, if an excess of `msg.value` is sent for the cross-chain fee, then the excess is refunded.

Both of these refund cases are not accounted for in the Claimer contract and therefore will become trapped, because the `Claimer` can't even sweep it's own balance due to the `msg.value < totalOfFees` check.

RECOMMENDATION

Consider adding a `rescueNative` function callable by the owner to the `Claimer` contract. Or consider refunding the entire Ether balance of the `Claimer` contract at the end of the `finalizeWithdrawals` execution, instead of trying to keep track of the refunds.

RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/287>

LOW **L-02**

ACKNOWLEDGED

Incorrect Refund Amount in `_completeWithdrawal`

CATEGORY	Logical Error
LOCATION	CollateralManager.sol
REVIEW	Remediation Review

DESCRIPTION

The refund logic in `_completeWithdrawal` is intended to return any excess native tokens sent along with the transaction. These tokens cover LayerZero's OFT messaging fees during cross-chain withdrawals.

When no OFT transaction occurs, the full `msg.value` is correctly refunded to the caller. However, when an OFT transaction is executed, the refund amount is calculated as `msg.value - (messagingFee.nativeFee + amountLD)`.

The issue arises because `amountLD` is not part of `msg.value`, it is provided by the collateral manager by unwrapping WUSDe to USDe. Consequently, any excess native tokens sent to `finalizeWithdraw` or `rescueFunds` are not refunded, since the calculation incorrectly subtracts `amountLD` from `msg.value`. This results in excess native tokens getting stuck in `CollateralManager`.

RECOMMENDATION

Adjust the refund calculation to subtract only the `messagingFee.nativeFee` from the `msg.value`.

RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/288>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Unnecessary MSTORE Operation

CATEGORY	Gas Optimization
LOCATION	VerifierLib.sol
REVIEW	Remediation Review

DESCRIPTION

In the `_verifyEip1271Signature` function the mstore operation after the external call was determined successful sets the length of the `returnData` to the `bytesToCopy`. However this length has already been set in the declaration of the `returnData` bytes so this operation is redundant and has no effect.

RECOMMENDATION

Consider removing the `mstore(returnData, bytesToCopy)` inside the success block as it is a wholly unnecessary gas expenditure.

RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/287>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.